

Curso Integrador I

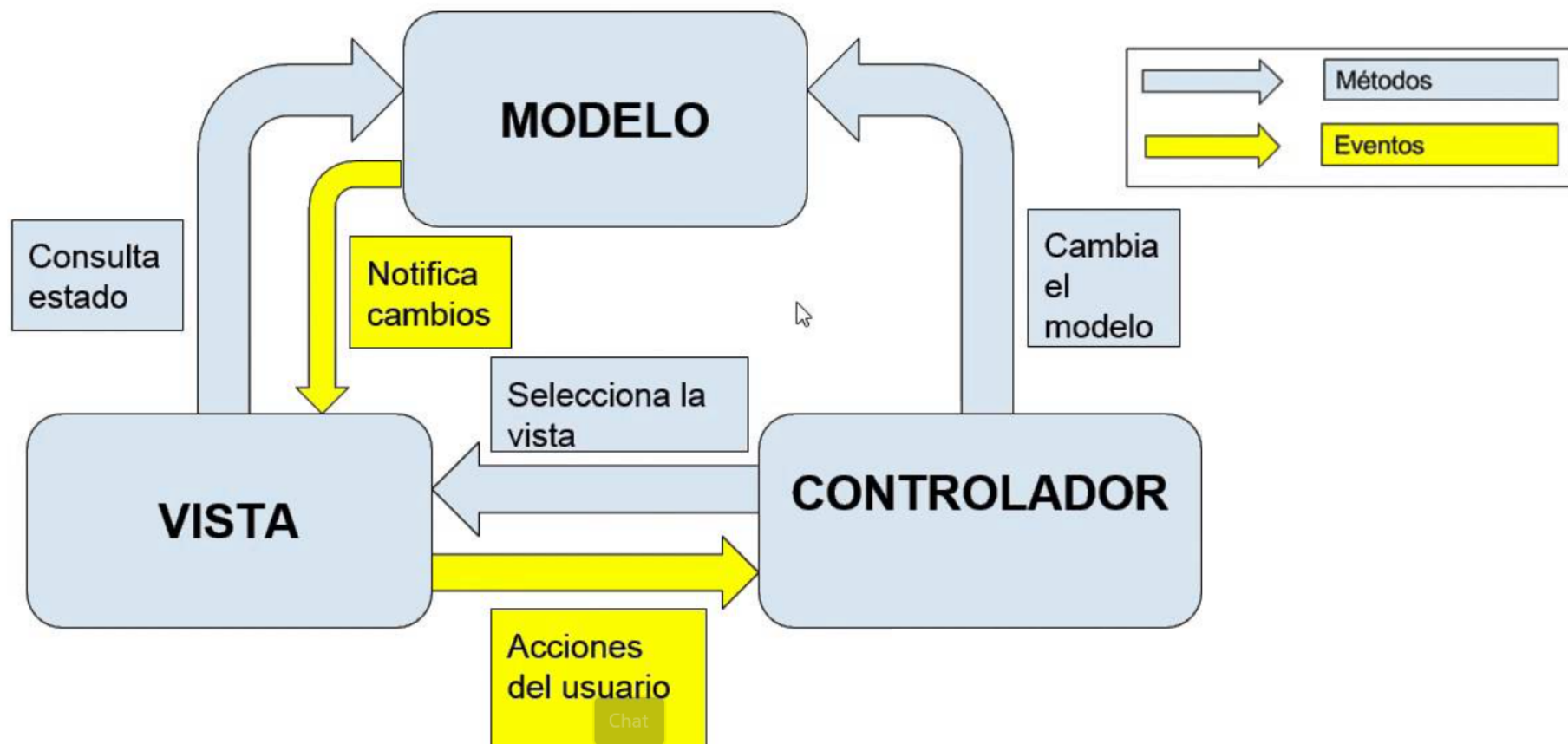
Data Access Object (DAO)

Semana 09 - Sesión 2

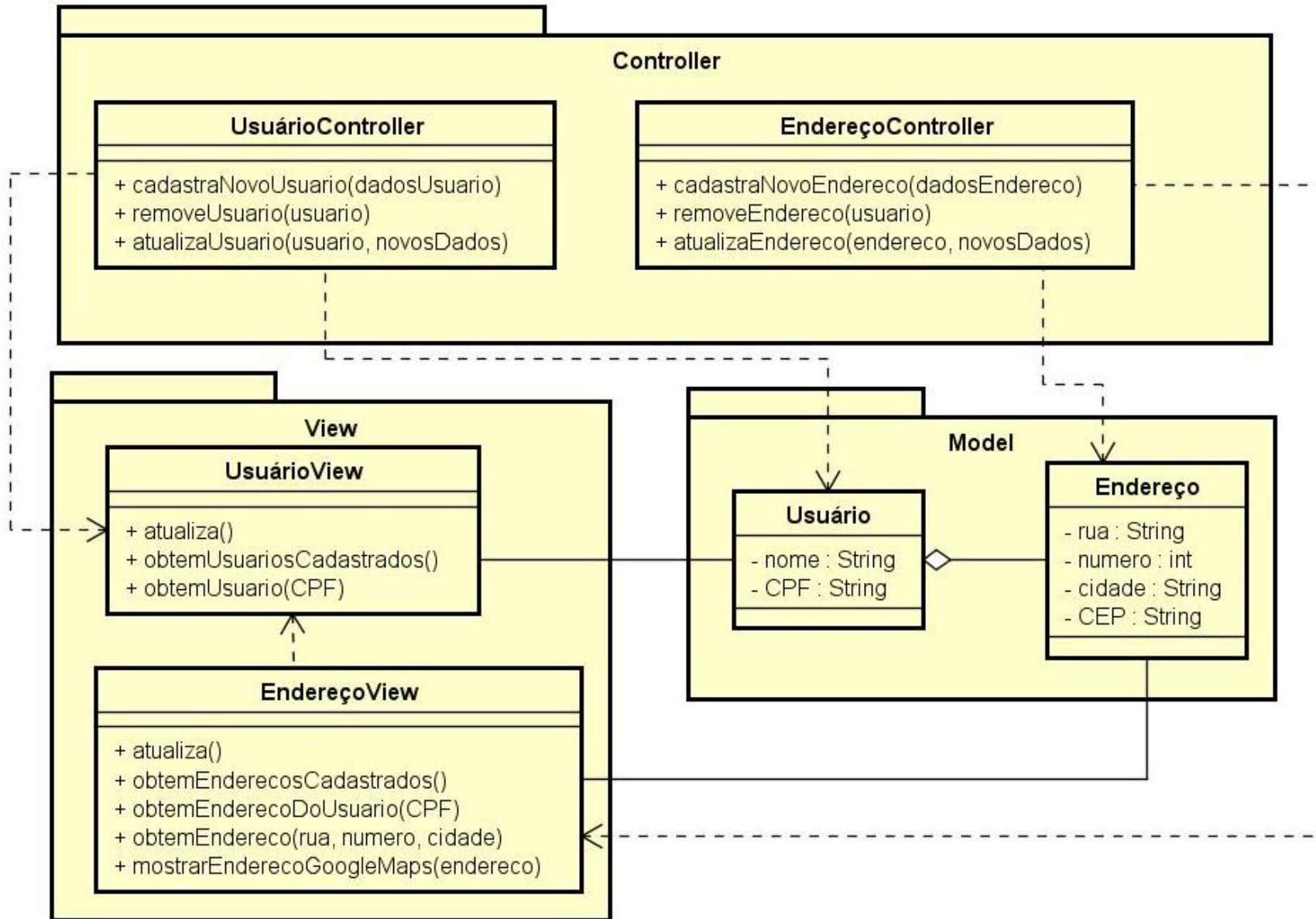
Desaprende lo que te limita

Conocimientos Previos

MVC



Dinâmica



Que observamos en la siguiente figura?

aprende lo que te limita

Logro de la Sesión

Al finalizar la sesión, el estudiante conoce el patrón de diseño DAO y lo aplica para complementar un diseño MVC en un proyecto de software.

CONTENIDOS

01

Patrón Modelo DAO

02

Componentes

03

Esquema con MVC

04

Ejercicios

Utilidad



- Permiten tener el código bien organizado, legible y mantenible, además te permite reutilizar código y aumenta la escalabilidad de proyecto.
- Propone separar por completo la lógica de negocio de la lógica para acceder a los datos

Problemas en el acceso a los datos



- El acceso a los datos varía según la fuente de los datos.
 - Tipo de almacenamiento (bases de datos relacionales, bases de datos orientadas a objetos, bases de datos noSQL, archivos planos, etc.)
 - Implementación de un proveedor particular para un tipo.
- Cuando los componentes del negocio necesitan acceder a una fuente de datos, pueden usar la API adecuada para lograr conectividad y manipular la fuente de datos.
- Problema: **Acoplamiento entre los componentes y la implementación de la fuente de datos:** aparece cuando se incluye el código de acceso a datos y conectividad proporcionado por diferentes API dentro de los componentes del negocio (objetos de dominio)
 - Estas dependencias de código en los componentes dificultan la migración de la aplicación de un tipo de fuente de datos a otro.
- Los componentes deben ser transparentes para el motor de persistencia o la implementación de la fuente de datos para facilitar la migración a diferentes productos de proveedores, diferentes tipos de almacenamiento y diferentes tipos de fuentes de datos.

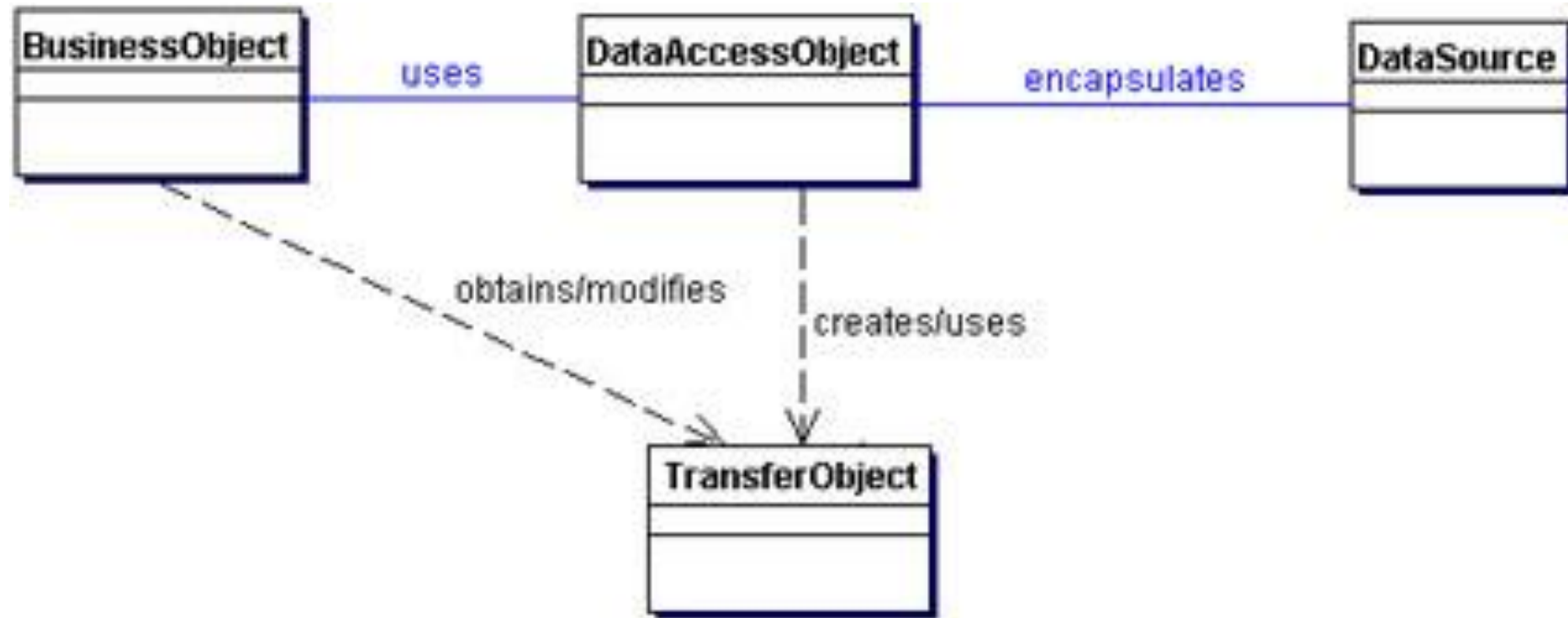
Desaprende lo que te limita

El patrón DAO (Objeto de acceso a datos)



Universidad
Tecnológica
del Perú

- Intención: Abstrae y encapsula todo el acceso a la fuente de datos



Desaprende lo que te limita

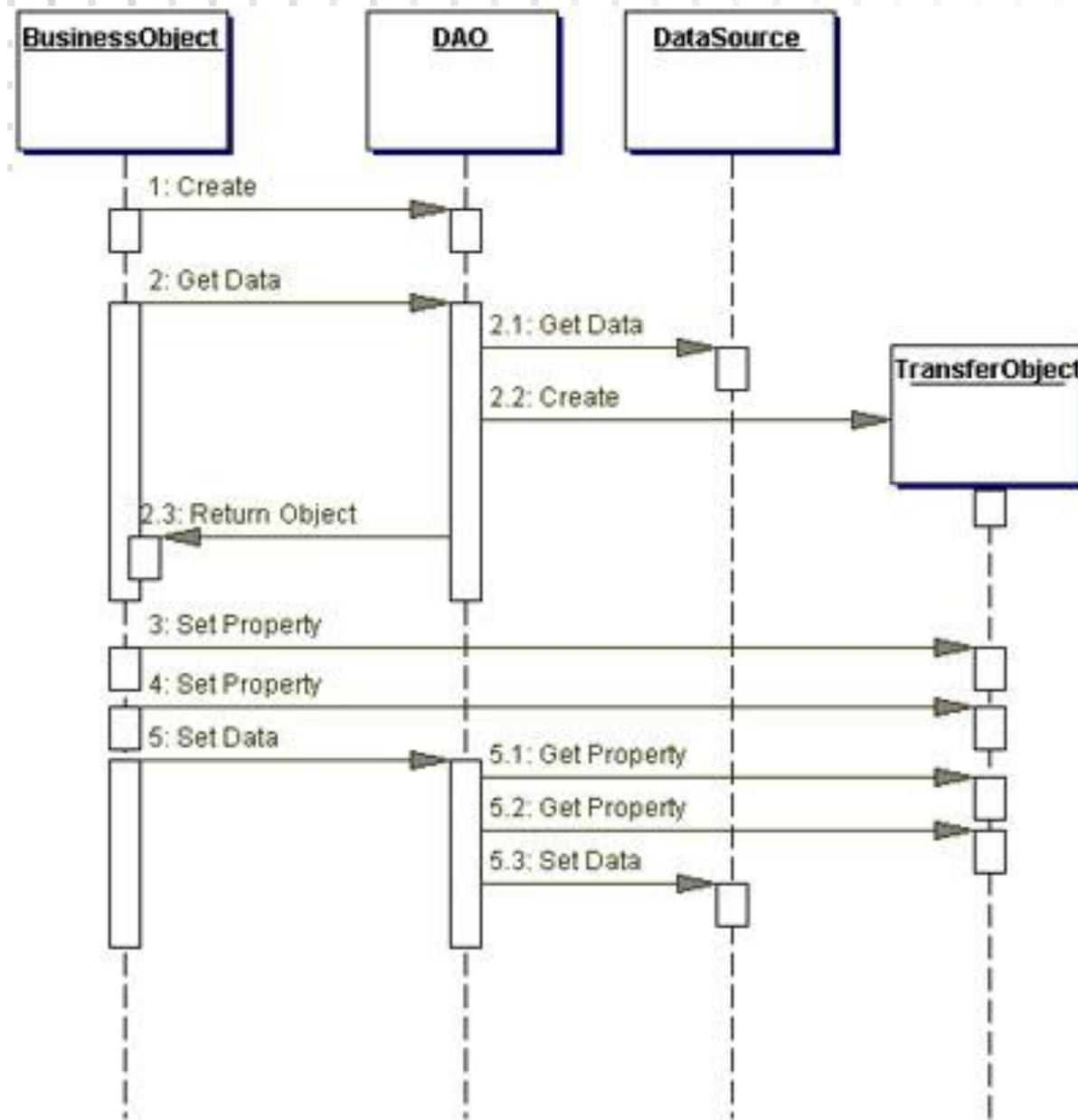
DAO - Participantes y responsabilidades



- **BusinessObject:** representa un objeto de negocio (POJO). Es el objeto que requiere acceso a la fuente de datos para obtener y almacenar datos.
- **DataAccessObject:** el objeto principal de este patrón. Abstrae la implementación de acceso a datos subyacente para el BusinessObject para permitir un acceso transparente a la fuente de datos. El BusinessObject también delega la carga de datos y las operaciones de almacenamiento en DataAccessObject.
- **DataSource:** representa una implementación de una fuente de datos. Una fuente de datos podría ser una base de datos como RDBMS, OODBMS, repositorio XML, sistema de archivos planos, etc.
- **Transfer Object :** (DTO) Utilizado como soporte de datos. El DataAccessObject puede utilizar un Transfer Object para devolver datos al cliente. El DataAccessObject también puede recibir los datos del cliente en un DTO para actualizar la información en la fuente de datos.

Desaprende lo que te limita

DAO



rende lo que te limita

DAO

¿De dónde obtenemos un DAO?

- Estrategias para obtener DAO's:
 - Estrategia ORM
 - Fábrica

Estrategia ORM (Object Relational Mapping)

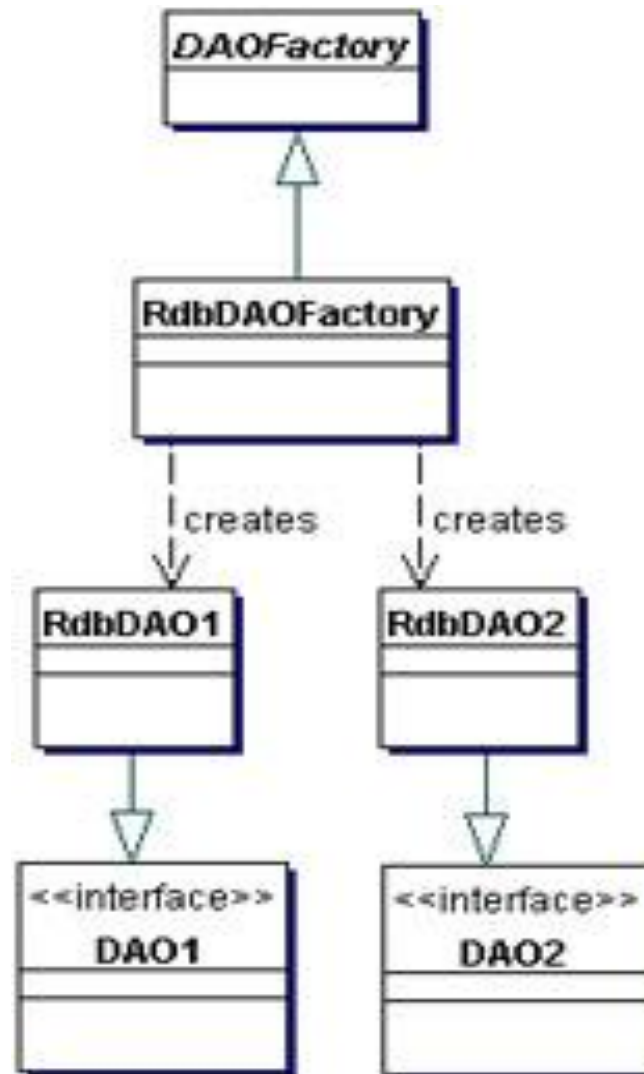


- Dado que cada BusinessObject corresponde a un DAO específico, es posible establecer relaciones entre BusinessObject, DAO y las implementaciones (como las tablas en un BD).
- Una vez que se establecen las relaciones, es posible escribir una utilidad de generación de código específica para aplicación que se esta construyendo que genera el código para todos los DAO requeridos por esta..
- Los metadatos para generar el DAO:
 - puede provenir de un archivo descriptor definido por el desarrollador.
 - o, alternativamente, el generador de código puede realizar una introspección automática de la base de datos y proporcionar los DAO necesarios para acceder a la base de datos.
- Si los requisitos para DAO son lo suficientemente complejos, considere usar **herramientas de terceros que proporcionan mapeo de objeto a relacional** para bases de datos RDBMS.
 - Ejemplos: API de persistencia de Java (JPA): Hibernate.
 - Eloquent, Doctrine (PHP); OR (NET)

Fábrica

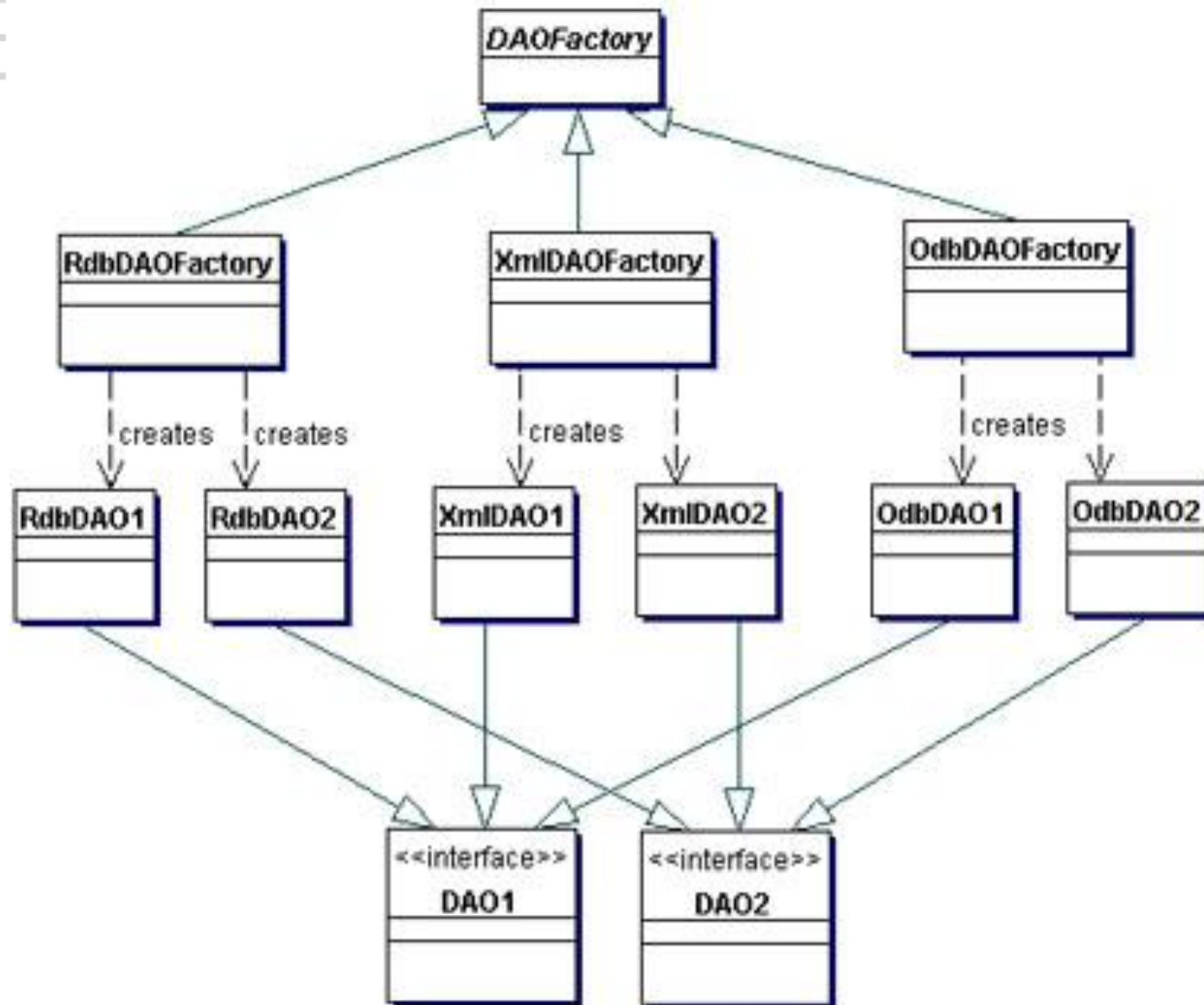
- El patrón DAO se puede hacer altamente flexible adoptando los patrones Abstract Factory [GoF] y Factory Method [GoF]
- Utilizar **Método fábrica**: Cuando el almacenamiento subyacente no está sujeto a cambios de una implementación a otra se puede utilizar el método fábrica para producir una serie de DAO necesarios para la aplicación.
- Utilizar **Fábrica abstracta**: Cuando el almacenamiento subyacente está sujeto a cambios de una implementación a otra, esta estrategia puede implementarse utilizando el patrón Abstract Factory. En este caso, esta estrategia proporciona un objeto DAO fabrica abstracta (Abstract Factory) que puede construir varios tipos de fábricas DAO concretas, cada fábrica admite un tipo diferente de implementación de almacenamiento persistente. Una vez que obtiene el DAO concreto de la fabrica para una implementación específica, se utiliza para producir DAO que soporten e implementen el acceso para ese almacenamiento específico.

DAO con fábrica



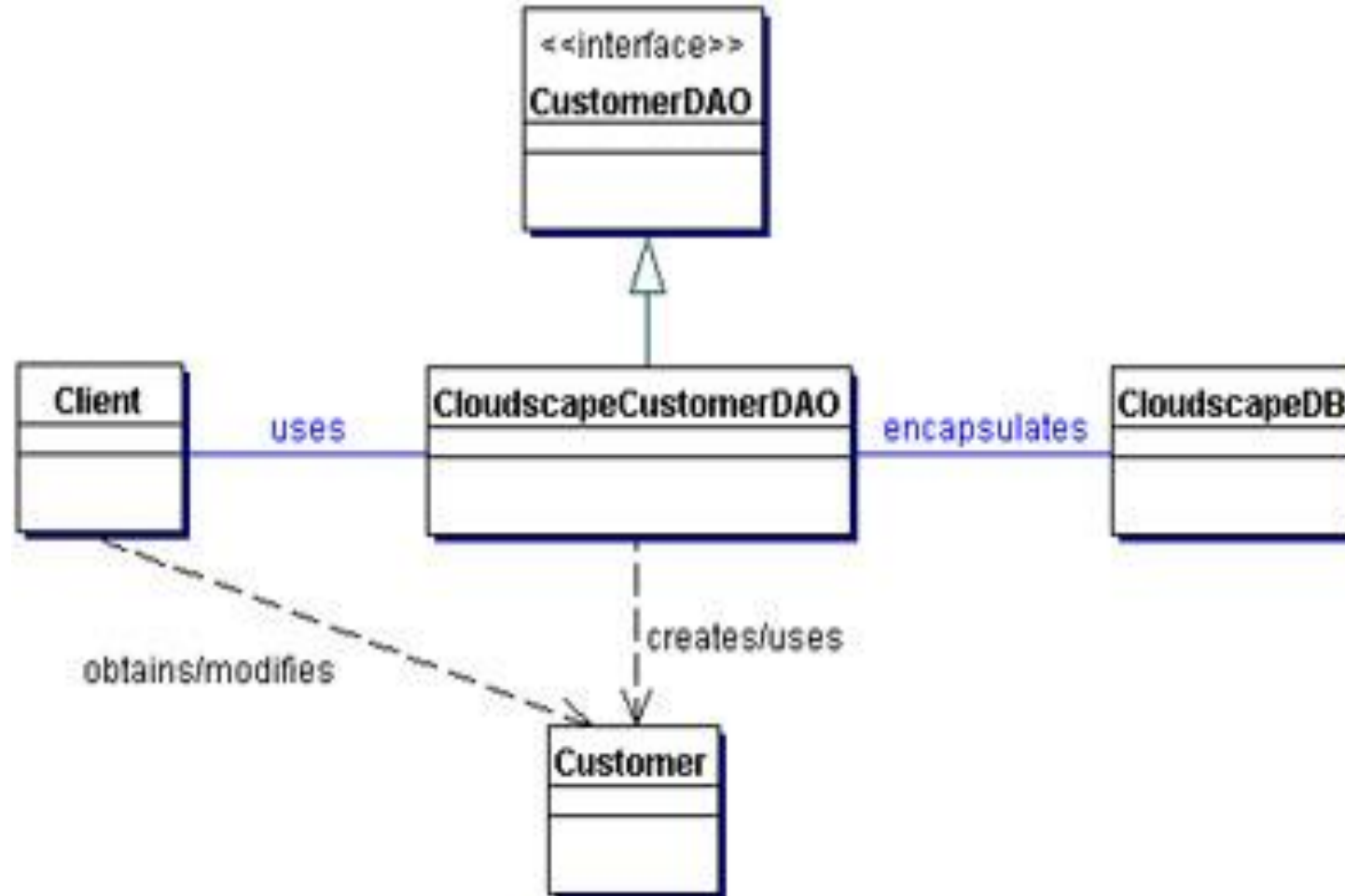
esaprende lo que te limita

DAO con Abstract Factory



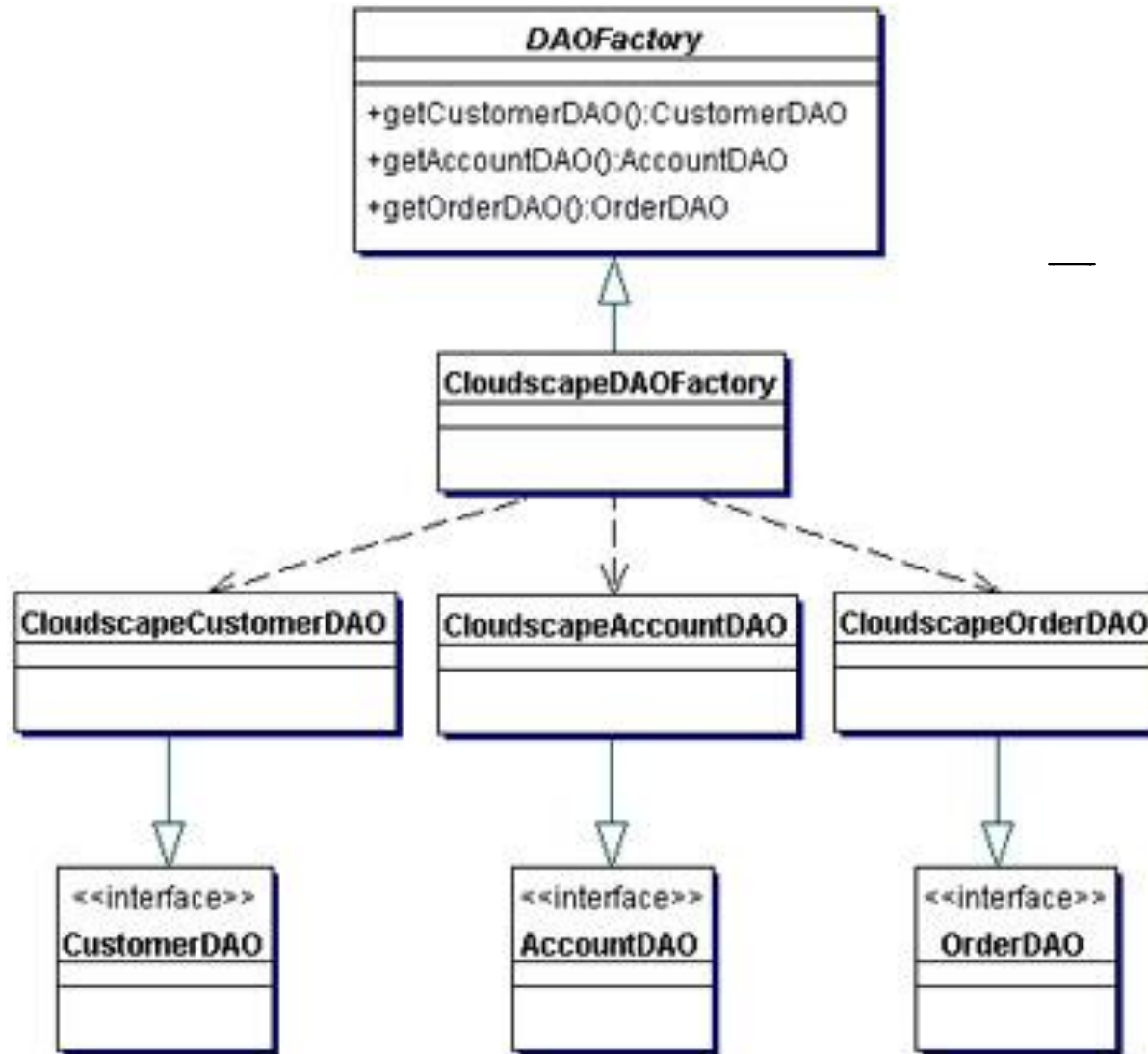
lo que te limita

Ejemplo



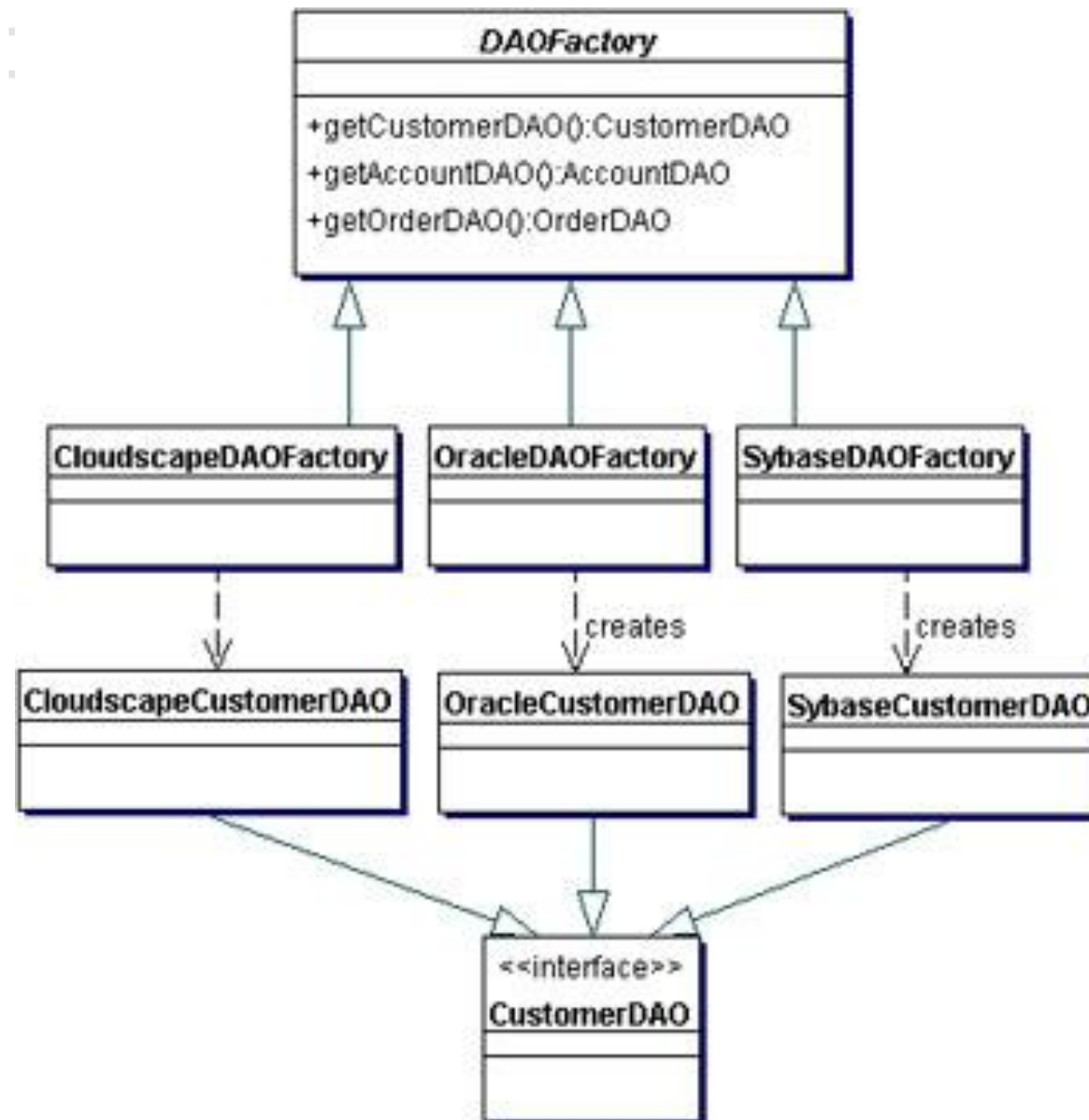
Desaprende lo que te limita

Ejemplo: uso del método fábrica



rende lo que te limita

Ejemplo: Uso de Abstract Factory



nde lo que te limita

DAO y otros patrones

DAO es un patrón general y complejo.

Martin Fowler, Patrones de arquitectura de aplicaciones empresariales

<http://martinfowler.com/eaCatalog/index.html>.

Alternativas Sencillas

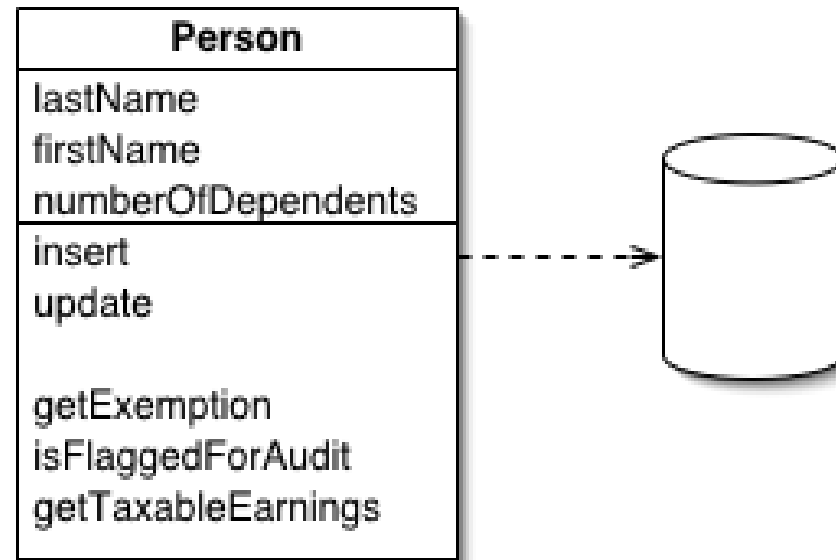
Table Data Gateway (Puerta de enlace de datos de tabla)

Row Data Gateway (Puerta de enlace de datos de fila)

Active Record/Active Domain Object (Registro activo / Objeto de dominio activo)

Patrones sencillos

Active Record/Active DomainObject



Un objeto que envuelve una fila en una tabla de base de datos, encapsula el acceso a la base de datos y agrega lógica de dominio a esos datos.

Características:

Los objetos del dominio dependen del mecanismo de persistencia

Se puede usar solo si el modelo del Dominio es simple y el mecanismo de persistencia no cambia.

Desaprende lo que te limita

Ejemplo Practico



Desaprende lo que te limita

Que hemos aprendido hoy



- Habilita la transparencia
- Permite una migración más sencilla
- Reduce la complejidad del código en los objetos de negocio.
- Agrega una capa adicional
- Necesita diseño de jerarquía de clases

Desaprende lo que te limita



Excelente tu
participación

No hay nada como
un reto para sacar lo
mejor de nosotros.



Ésta sesión
quedará
grabada para tus
consultas.



PARATI

1. Sigue practicando,
vamos tu puedes!! .
2. No olvides que
tienes un FORO
para tus consultas.

Desaprende lo que te limita



Universidad
Tecnológica
del Perú



**Universidad
Tecnológica
del Perú**

Desaprende lo que te limita